

PROJECT REPORT: DOWNLOADJANITOR

Day 01 of the 21 Days of Vibecoding Challenge — Automated File System Assistant

Author: Devaki Harish Nair (2nd Year CSE)	Target OS: Windows 10/11
Repository: github.com/devakihnair/21days	Core Stack: Python 3.14, Watchdog, Colorama

1. Base Idea & Problem Statement

In academic and everyday software engineering workflows, the local **Downloads** directory is notorious for becoming an unmanageable digital landfill. College students constantly download syllabus PDFs, lecture slides, research papers, assignment code snippets, installation setups, and compressed assets.

Manual file sorting is tedious, error-prone, and quickly abandoned. Most existing file organizers either lack safety guarantees (accidentally overwriting existing files with identical names or interrupting in-progress browser downloads) or require complex configurations. **DownloadJanitor** solves this by delivering an automated, non-destructive, and undoable file sorting engine with zero cognitive friction.

2. Technology Stack & Architecture

Component	Technology / Tool	Role / Architecture Rationale
Runtime	Python 3.14 (Virtual Environment)	Cross-platform portability, rich standard library, and rapid iteration.
File System Engine	pathlib & shutil	High-level object-oriented filesystem access and atomic file movement.
Real-Time Watcher	watchdog (v6.0.0)	OS event-driven notification hooks (Windows ReadDirectoryChangesW) instead of CPU-heavy polling.
Terminal UI	colorama (v0.4.6)	Cross-platform ANSI color formatting and intuitive interactive REPL.
1-Click Launcher	Windows Batch (.bat)	Provides desktop shortcut execution without terminal navigation.

3. Core Working Mechanism & Safety Pillars

DownloadJanitor is built on four core architectural pillars:

A. Deterministic Categorization: Files are mapped to specific semantic directories based on extension matrices:

- **Documents:** .pdf, .docx, .pptx, .xlsx, .txt, .csv, .md
- **Images:** .png, .jpg, .jpeg, .webp, .svg, .gif
- **Code:** .py, .ipynb, .cpp, .c, .java, .html, .css, .js, .sql, .rs, .dart
- **Installers:** .exe, .msi, .msix, .iso, .apk
- **Media:** .mp4, .mkv, .mp3, .wav, .mov
- **Archives:** .zip, .rar, .7z, .tar.gz

- B. Collision-Free Renaming:** Overwriting files is mathematically prohibited. If a file named notes.pdf already exists at destination, the engine probes incremental indices notes (1).pdf, notes (2).pdf, guaranteeing complete data preservation.
- C. Download-in-Progress Filtering:** Browser temporary downloads (.crdownload, .part, .tmp) are automatically skipped, preventing partial or corrupt file transfers.
- D. Transactional Undo Stack:** Every live organization run appends an atomic manifest of (source_path, moved_path) tuples to .janitor_history.json. The --undo command pops the latest transaction from the LIFO stack and restores all files back to their exact original positions.

4. Computer Science Concepts & Design Patterns

For academic study and interviews, DownloadJanitor serves as an ideal reference implementation for several foundational software principles:

- 1. The Observer Design Pattern:** In Live Watch Mode (Option 4), the application uses OS filesystem hooks to capture on_created and on_modified event dispatches, eliminating CPU polling loops.
- 2. LIFO Stack Transaction Management:** The undo engine implements a persistent Last-In, First-Out ledger, mimicking database rollback journals.
- 3. Idempotency:** Running the organization pass multiple times leaves the filesystem in a stable, identical state without re-creating nested duplicate structures.
- 4. REPL CLI Architecture:** The interactive menu implements a safe state-machine enabling dynamic target directory switching without restarting the process.

5. Verification & Final Outcomes

The system underwent rigorous dual-phase validation:

Validation Phase	Test Scope	Outcome / Performance
Phase 1: Sandbox Validation	19 generated synthetic files (including mock .crdownload & .tmp)	100% of candidate files sorted. In-progress files properly ignored. Dry-run preview matched actual execution. Complete undo verified with zero data corruption.
Phase 2: Production Benchmark	251 real student files in local Downloads directory (KTU syllabus notes, installers, datasets, media)	Processed 251 files in <0.45 seconds: 127 Documents, 68 Images, 33 Installers, 14 Archives, 7 Media files, 2 Others. Memory footprint <22 MB RAM.

6. Project Status & Roadmap Connection

DownloadJanitor successfully concludes **Day 01 of the 21-Days Vibecoding Challenge**. The source code, documentation, test suite, and launcher scripts are committed and publicly hosted on GitHub at github.com/devakihnair/21days.

Next in Sequence: Day 02 transitions into Generative AI with **ExamPrep AI**, a Python application that accepts lengthy lecture PDFs or syllabus slides and automatically synthesizes high-yield revision flashcards and practice exam quizzes.